TutorTrace: Classifying Learner States from Replayed AI-Assisted Programming Sessions

David Barron, et al. -- PRIME Lab, Department of Computer Science, Virginia Tech

UIST '26, October 2026. Camera-ready version.

Abstract

Students in programming courses now work alongside AI assistants, and the moments when they need a person are hidden inside long sessions of editing, running and prompting. We present TutorTrace, a dataset and taxonomy for classifying learner states from replayed sessions that combine editor events, run results and assistant prompts. From a live deployment in a data structures course (87 students, two lab sections), three trained raters labeled 2,340 thirty-second windows into six states, including a stalled-after-error state that separates unproductive stalling from exploratory tinkering. Agreement was Fleiss' kappa 0.71. A gradient-boosted classifier over telemetry features reached macro-F1 0.78 on held-out students and 0.69 on a second course after per-course normalization. We describe the replay-based labeling interface, release the labeling guide and data under per-stream consent, and discuss how such signals should prompt human check-ins rather than automatic interventions.

1 Introduction

An instructor walking around a lab sees a student staring at a screen. Is the student thinking, or stuck? With AI assistants in the loop, the question has become harder: a student may be productively testing an assistant's suggestion, or cycling through the same request with growing frustration. Neither the assistant nor the autograder sees the whole session.

TutorTrace makes the whole session visible. We contribute (1) a replayable dataset of AI-assisted programming sessions from a live deployment, (2) a six-state taxonomy with a labeling guide and reliability evidence, and (3) a classifier and a cross-course analysis that show what transfers and what does not.

2 Related Work

Process data from programming environments has been used to detect struggle and to support instructors through session replay. Closest to our work, Ostrowski et al. labeled thirty-second windows of LLM-assisted CS1 sessions into four states and found repeated near-identical prompts to be the strongest predictor of stalling. Our taxonomy separates stalled-after-error from general stalling and from exploratory tinkering, and we evaluate transfer to a second course. Work on productive failure reminds us that some struggle is valuable; our goal is to find the struggle that is not.

3 Data

3.1 Deployment

We deployed an instrumented browser editor with an assistant side panel in CS 3114 (Data Structures and Algorithms) in Fall 2025. The labeled session took place in two lab sections led by three TAs. The editor logged insertions, deletions, pastes, runs with results, and assistant exchanges with millisecond timestamps.

3.2 Telemetry gap

A websocket reconnect bug dropped roughly 12% of events in one section until a hotfix was deployed during the lab. We excluded every window overlapping the gap before labeling and release the list of excluded windows.

3.3 Data and consent

Consent separated three streams: editor events, run results and assistant prompts. Nine of 87 students declined prompt logging. The release contains derived features and labels for all consenting students and raw prompt text only for students who opted in to its release. Identifiers are replaced with per-release random ids and string literals are stripped from code snapshots. A datasheet is in the supplementary material.

4 Taxonomy and Labeling

State	Definition (abridged)	Share of windows
Productive progress	edits that move toward a passing run	31%
Exploratory tinkering	rapid small edits and runs, no reverts	18%
Stalled after error	no successful run in 3 min, reverted edits or idle after an error	14%
Assistant help-seeking	composing or reading an assistant exchange	17%
Verifying	running tests and inspecting output	13%
Disengaged	idle or off-task without a preceding error	7%

Raters labeled windows in a replay interface that shows the editor, a timeline with run and idle markers, and the assistant panel, with 30 seconds of preceding context. After two calibration rounds with guide v3, three raters labeled all 2,340 windows independently.

Agreement	Fleiss' kappa
Overall (3 raters)	0.71
Assistant help-seeking	0.84
Verifying	0.77
Productive progress	0.73
Disengaged	0.66
Tinkering vs stalled-after-error	0.58

The tinkering/stalled boundary was hardest. Guide v3's rule for it raised agreement on that pair from 0.49 to 0.58.

5 Classifier

We trained gradient-boosted trees over 46 features per window: edit and run rates, time since last successful run, idle gaps, revert counts, prompt counts and prompt similarity to the previous prompt. All evaluation uses folds grouped by student.

Model	Macro-F1 (held-out students)
Majority class	0.08

Counts only (logistic regression)	0.64
Gradient boosting, all features	0.78

The most important features were time since the last successful run, prompt similarity to the previous prompt, and idle time following a failed run.

6 Cross-Course Analysis

To test generality we labeled 610 windows from an introductory Python course (Spring 2026, 41 students) with the same guide and raters, trained on CS 3114 and tested on the second course.

Test set	Macro-F1 raw	Macro-F1 normalized	F1 stalled-after-error
CS 3114, held-out students	0.78	0.78	0.71
Intro Python (Spring 2026)	0.64	0.69	0.52

Normalizing rate features per course recovered part of the gap. Stalled-after-error transferred worst, because its signature depends on the course's error messages and run habits.

7 Discussion

Telemetry can say that a student is probably stalled; it cannot say why. We recommend using predictions as prompts for a human check-in, with replay as a second stage for flagged sessions. Assistant prompts were the most sensitive stream for students; derived labels may be enough for most uses.

8 Limitations

Two courses at one institution; labels are judgments even at kappa 0.71; states depend on the assistant students used; the telemetry gap shows that infrastructure failures shape datasets.

References

[1] Ostrowski, H., Lindqvist, M., Bello, A., and Chandrasekar, R. Productive struggle or silent stalling? Telemetry signals of help-seeking in LLM-assisted CS1. L@S 2025, 112-123.

[2] Kapur, M. Productive failure. Cognition and Instruction 26, 3 (2008), 379-424.

[3] Nakamura, R. and Feld, S. Labeling learner states in programming sessions. ICER 2025, 233-245.

[4] Duarte, P. and Sim, H. Beyond one course: do behavioral-state classifiers transfer across programming courses? EDM 2025.

[5] Brennan, C., Zhou, W., and Adler, R. What will students share? arXiv:2602.11873, 2026.